INFORMATIK LK · WELT 0 · DS 2–7

Die Akademie

Basics, Reihungen, eigene Methoden



L0.2 ····· L0.3 ····· L0.4 ····· L0.5 ····· PC 0 ····· BOSS

Die Akademie im Überblick



DS 2

IDE-Level 0.2

Werkstatt

Datentypen, Operatoren, Verzweigung, Schleifen

Lab-Level 0.2



DS 3

IDE-Level 0.3

Vorratskammer

Reihungen, Referenzen, Zeichenketten

Lab-Level 0.3



DS 4–5

IDE-Level 0.4 + 0.5

Zauberschule

eigene Methoden: Signatur, Parameter, Rückgabe

Lab-Level 0.4 + 0.5



DS 6

Papier-Compiler 0

Methoden-Werkstatt

Rezept, Fehlergalerie, Abi-Format

Lab-Level 0.6



DS 7

Boss 0

Aufnahmeprüfung

Abi auf Zeit: ≈ 24 BE in 60 min

Lab: Online-Boss

Warum so viel Methoden? Im Abitur heißt es immer: „Implementieren Sie die Methode …“ – mit vorgegebener Signatur.

Datentypen und Variablen

Abi-Typname	Java	Beispielwerte
Ganzzahl	`int`	`42` `-7` `0`
Kommazahl	`double`	`3.5` `-0.25` `2.0`
Wahrheitswert	`boolean`	`true` `false`
Zeichen	`char`	`'A'` `'7'` `' '`
Zeichenkette	`String`	`"Aria"` `""`
nichts	`void`	keine Rückgabe (ab DS 4)

Achtung: Zeichen in einfache `'A'`, Zeichenketten in doppelte Anführungszeichen `"A"`. Kommazahlen mit Punkt: `3.5`.

Variable = beschriftete Kiste



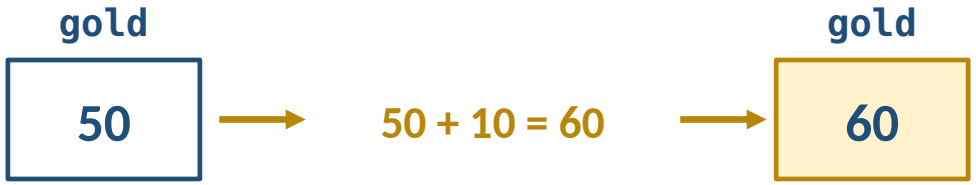
Zuweisung: erst rechts rechnen, dann links ablegen

```
gold = gold + 10;
```

Java

gold ← gold + 10

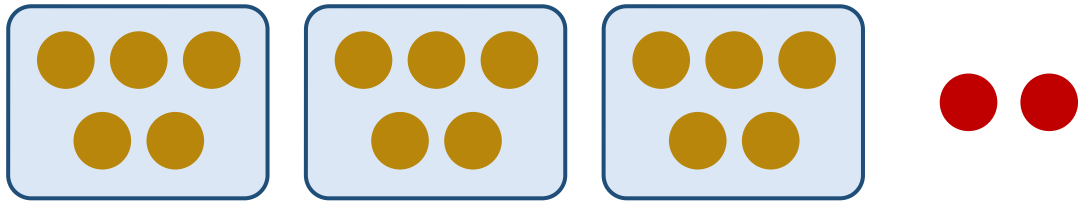
Struktogramm (EH §5)



Merke: **=** ist keine Gleichung, sondern „bekommt den Wert“. Im Struktogramm: ←

Rechnen mit Ganzzahlen: / und %

17 Tränke, 5 passen in eine Kiste



`17 / 5` = **3 volle Kisten** **`17 % 5`** = **2 übrig**

	Java	Struktogramm
Vergleich	`==` `!=` `<` `<=` `>` `>=`	`=` `≠` `<` `≤` `>` `≥`
Logik	`&&` `||` `!`	und oder nicht

Achtung: `/` mit zwei Ganzzahlen schneidet ab: **`7 / 2`** ergibt 3. Gerundet wird nie, der Rest geht verloren.

Ausdruck	Ergebnis	Bedeutung
`17 / 5`	**3**	wie oft passt 5 hinein
`17 % 5`	**2**	Rest (mod)
`7 / 2`	**3**	ganzzahlig
`7.0 / 2`	**3.5**	Kommazahl-Division
`1234 % 10`	**4**	letzte Ziffer
`1234 / 10`	**123**	letzte Ziffer streichen
`-7 % 2`	**-1**	Vorzeichen bleibt!

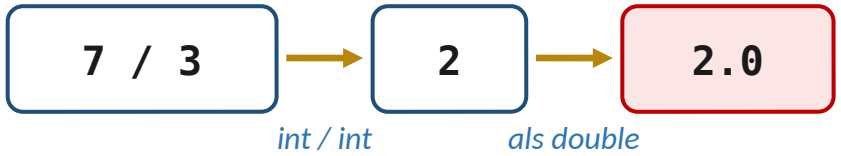
Denkanstoß: Wie viele Kisten brauchst du für 17 Tränke? **`17 / 5`** liefert 3 – eine zu wenig. Lösung ohne Math: Level 0.2.

Die Casting-Falle: Durchschnitt von 1, 2 und 4

```java
int a = 1, b = 2, c = 4;
double d1 = (a + b + c) / 3;              // 2.0
double d2 = (a + b + c) / 3.0;            // 2.3333333333333335
double d3 = (double) (a + b + c) / 3;     // 2.3333333333333335
double d4 = (double) ((a + b + c) / 3);   // 2.0
```

FALSCH
RICHTIG
RICHTIG
FALSCH

d1: zu früh



d2: richtig

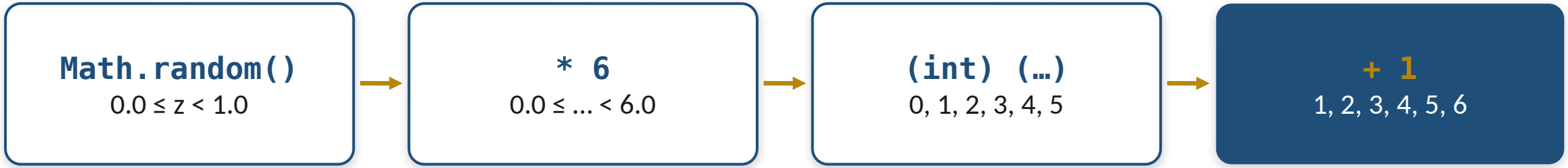


`(int) 3.9 → 3`
Cast zu int schneidet ab – er rundet nicht.

Merke: Mindestens ein Operand muss eine Kommazahl sein, **bevor** geteilt wird: **`3.0`** oder **`(double)`** vorn.

Achtung Abi: Ein Durchschnitt ist immer eine Kommazahl – auch wenn die Werte Ganzzahlen sind.

Zufallszahl im Bereich und ganzzahliges Runden




```
int wurf = (int) (Math.random() * 6) + 1;
```

allgemein für von … bis (beide inklusive):

```
(int) (Math.random() * (bis - von + 1)) + von
```

bis − von + 1 = Anzahl möglicher Werte

Falle: **`(int) Math.random() * 6`** castet zuerst – aus 0.xx wird 0, das Ergebnis ist immer 0. Klammern setzen!

Ganzzahlig runden (wert ≥ 0)

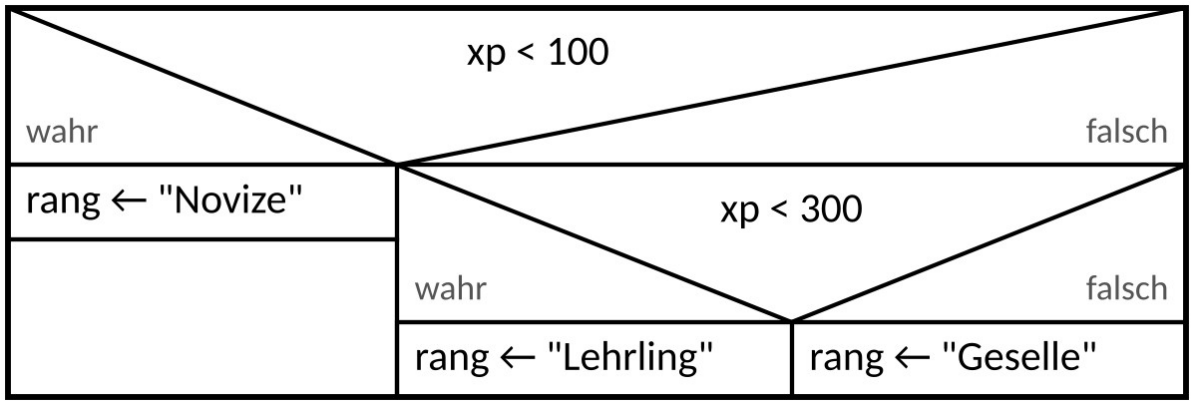
```
int r = (int) (wert + 0.5);
```

wert	wert + 0.5	(int)
2.4	2.9	**2**
2.5	3.0	**3**
2.9	3.4	**3**

EH 2025: beides gehört zum Abi-Handwerk – ohne **`Math.round`**.

Verzweigung: if, else if, else

```
String rang;
if (xp < 100) {
    rang = "Novize";
} else if (xp < 300) {
    rang = "Lehrling";
} else {
    rang = "Geselle";
}
```



else if = neue Verzweigung im falsch-Zweig

Bedingungen verknüpfen
`xp >= 100 && xp < 300`
Struktogramm: xp ≥ 100 und xp < 300
Reihenfolge zählt: Die erste wahre Bedingung gewinnt.

xp	rang
99	**Novize**
100	**Lehrling**
299	**Lehrling**
300	**Geselle**

Werte mit javac geprüft

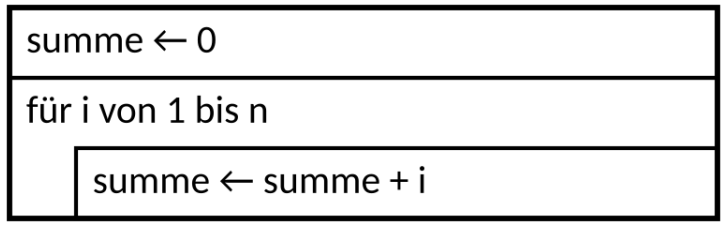
Grenzen testen: immer den Wert direkt davor und genau auf der Grenze.

Schleifen: for, while und do-while

`for`

Zählschleife: Anzahl bekannt

```
int summe = 0;
for (int i = 1; i <= n; i++) {
    summe = summe + i;
}
```

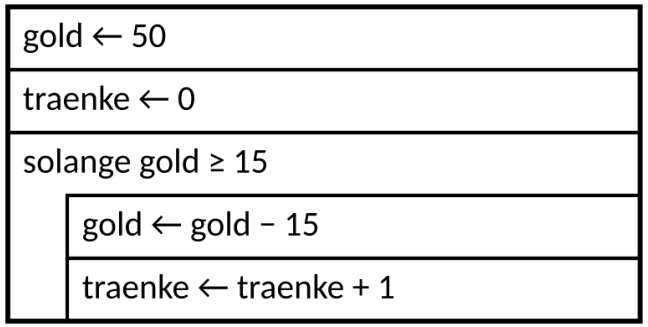


n = 5 → summe = 15

`while`

solange die Bedingung gilt

```
int gold = 50;
int traenke = 0;
while (gold >= 15) {
    gold = gold - 15;
    traenke++;
}
```



traenke = 3, gold = 5

`do-while`

Prüfung erst am Ende

```
int wurf;
do {
    wurf = (int) (Math.random()*6) + 1;
} while (wurf != 6);
```

läuft mindestens einmal

Selten nötig. Im Struktogramm nutzen wir die kopfgesteuerten Formen: „für …“, „für jedes …“, „solange …“.

endet erst mit wurf = 6

Trace: quersumme(1234) Schritt für Schritt

```
public static int quersumme(int n) {
    int summe = 0;   // → Tabellenzeile (Startwerte)
    while (n > 0) {
        summe = summe + n % 10;
        n = n / 10;   // → Tabellenzeile
    }
    return summe;
}
```

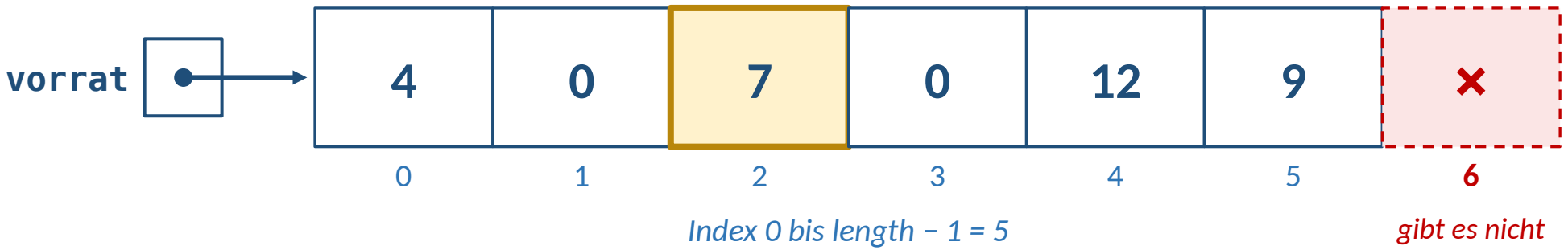
Durchlauf	n % 10	summe	n
vor der Schleife	–	0	1234
1	4	**4**	**123**
2	3	**7**	**12**
3	2	**9**	**1**
4	1	**10**	**0**

quersumme(n: Ganzzahl): Ganzzahl
summe ← 0
solange n > 0
▸ summe ← summe + n mod 10
▸ n ← n / 10 *// ganzzahlig*
gib summe zurück

gib 10 zurück 1 + 2 + 3 + 4 = 10

Tracetabelle: nur Variablen, keine Bedingungen; eine Zeile an jeder Stelle mit // → Tabellenzeile.

Die Reihung: ein Regal mit nummerierten Fächern




```
int[] vorrat = {4, 0, 7, 0, 12, 9};
int[] leer = new int[5];     // {0, 0, 0, 0, 0}
vorrat[2]                    // 7
vorrat[4] = 15;              // Fach 4 neu belegen
vorrat.length                // 6 – ohne Klammern!
vorrat[vorrat.length - 1]    // 9, das letzte Fach
int[] nix = new int[0];      // leer: length 0
```

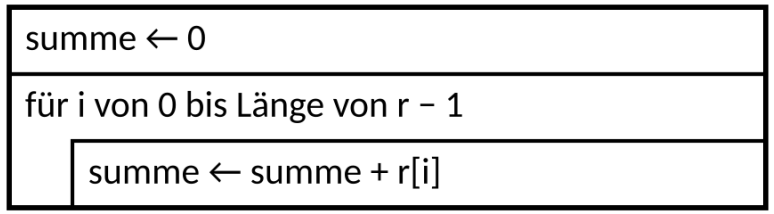
Abi-Sprache: „Reihung von Ganzzahl“, Index ab 0, „Länge von r“. Nie „Array“.

Achtung: `vorrat[6]` → ArrayIndexOutOfBounds-Exception, Absturz. Bei leerer Reihung gibt es nicht einmal `r[0]`.

Durch die Reihung laufen: for oder for-each

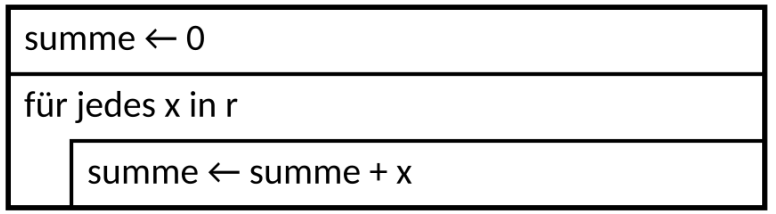
for mit Index

```
int summe = 0;
for (int i = 0; i < r.length; i++) {
    summe = summe + r[i];
}
```



for-each

```
int summe = 0;
for (int x : r) {
    summe = summe + x;
}
```



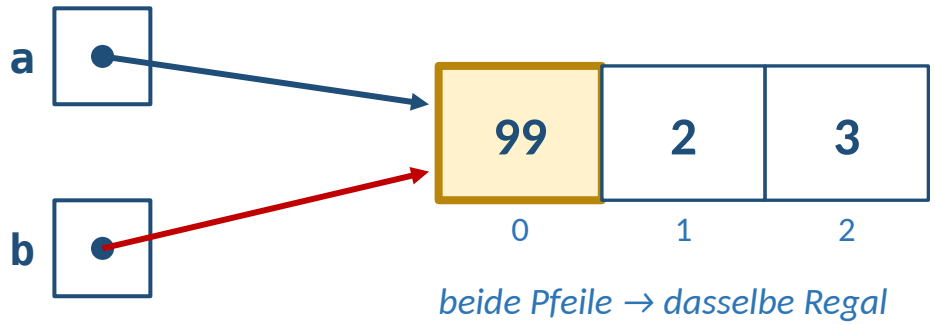
Für den Vorrat
liefern beide 32

	for mit Index	for-each
kennt den Index i	**ja**	**nein**
Fächer verändern (r[i] = …)	**ja**	**nein – x ist eine Kopie**
Teilbereich, z. B. ab Index 1	**ja**	**nein**
typische Falle	`i <= r.length → Absturz`	`x = 0; ändert r nicht`

Zwei Schilder, ein Regal: Referenz statt Kopie

int[] b = a; kopiert nur den Pfeil

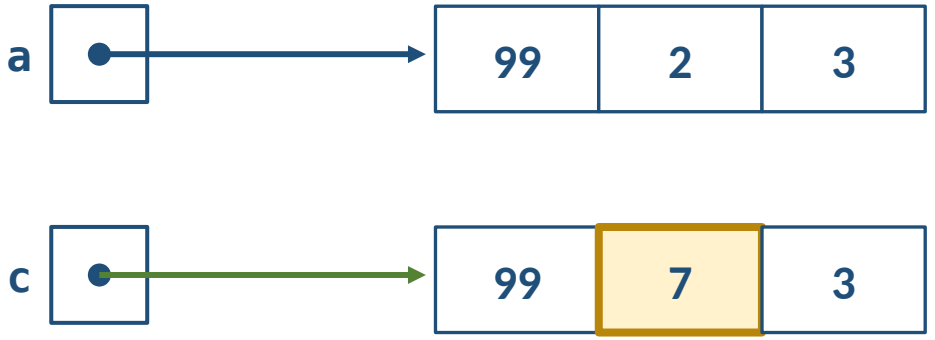
```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
println(a[0]);   // 99
```



beide Pfeile → dasselbe Regal

Echte Kopie: neues Regal, Werte einzeln

```
int[] c = new int[a.length];
for (int i = 0; i < a.length; i++) {
    c[i] = a[i];
}
```



c[1] = 7 ändert nur c

Merke: Eine Reihungsvariable enthält nicht das Regal, sondern eine Referenz (einen Pfeil) darauf. Eine Kopie entsteht nur mit **`new`**.

Zeichenketten: was im Abitur erlaubt ist

Beispiel: `String s = "Feuerball";`

Operation (EH)	Java	Ergebnis
Länge	`s.length()`	**`9`**
Zeichen an Position	`s.charAt(0)`	**`'F'`**
Teilzeichenkette	`s.substring(0, 5)`	**`"Feuer"`**
Verketten	`s + "!"`	**`"Feuerball!"`**
Gleichheit	`s.equals("Feuerball")`	**`true`**
enthält Teilzeichenkette	`s.contains("ball")`	**`true`**
lexikographischer Vergleich	`"Aria".compareTo("Brann")`	**`-1 (davor)`**
ASCII-Wert ↔ Zeichen	`(int) 'A'  (char) 98`	**`65  'b'`**

Nicht verwenden

- `toLowerCase()`
- `toUpperCase()`
- `split(…)`
- `indexOf(…)`
- `trim()`
- `replace(…)`

Vergleich: **`equals`**, nie **`==`** bei Zeichenketten.

Tipp: Einzelne Zeichen (char) vergleicht man mit **`==`**, z. B. **`s.charAt(i) == 'a'`**.
Großbuchstabe? **`c >= 'A' && c <= 'Z'`**

Anatomie einer Zauberformel (Methode)

Name = der Spruch | **Zutaten = die Parameter** | **Wirkung = die Rückgabe**

```
public static int schutzwert (int ruestung, int stufe) {
```

- `public static` — Sichtbarkeit + static (Klassenmethode)
- `int` — Rückgabetyp = Wirkung
- `schutzwert` — Name
- `(int ruestung, int stufe)` — Parameterliste: Typ und Name je Zutat

```
    int wert = ruestung + 5 * stufe;
    return wert;
}
```

← Rumpf: lokale Variable

← liefert den Wert und beendet die Methode

Abi-Signatur

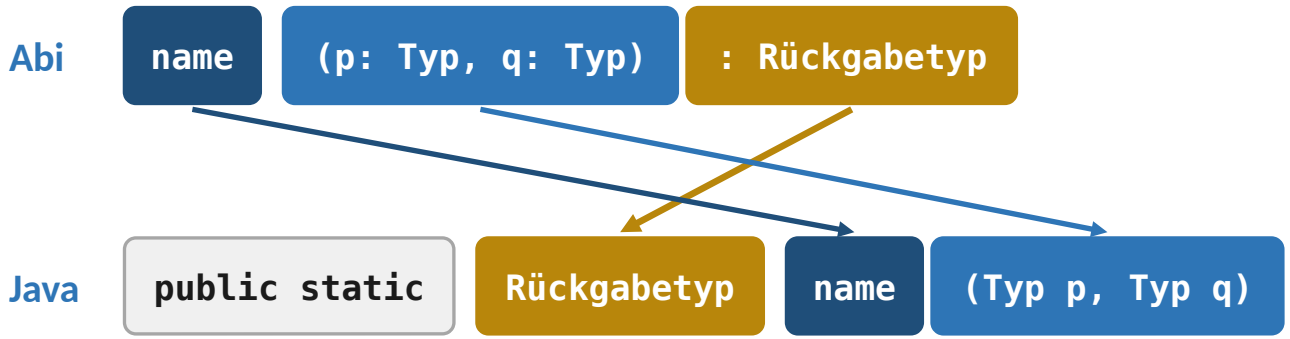
schutzwert(ruestung: Ganzzahl, stufe: Ganzzahl): Ganzzahl

Aufruf

`schutzwert(12, 3)` → 27

Abi-Signatur → Java-Methodenkopf

Abi-Notation	Java
Ganzzahl	`int`
Kommazahl	`double`
Wahrheitswert	`boolean`
Zeichen	`char`
Zeichenkette	`String`
Reihung von Ganzzahl	`int[]`
Reihung von Zeichenkette	`String[]`
nichts	`void`



Rückgabetyp nach vorn · in den Klammern Typ vor Name · Doppelpunkte weg

bonus(stufe: Ganzzahl, kritisch: Wahrheitswert): Kommazahl

```
public static double bonus(int stufe, boolean kritisch)
```

markiere(namen: Reihung von Zeichenkette, i: Ganzzahl): nichts

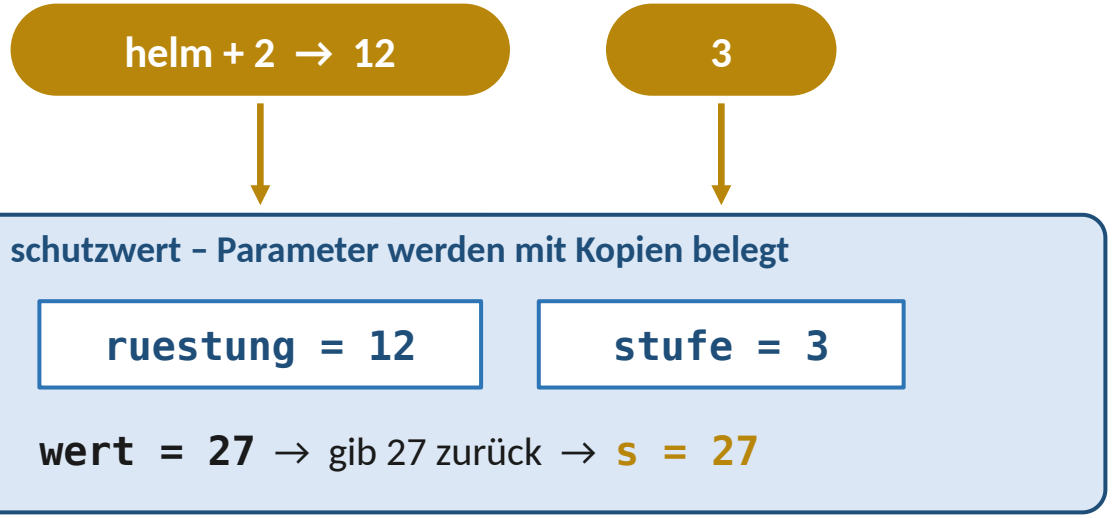
```
public static void markiere(String[] namen, int i)
```

Typische Fehler: Rückgabetyp vergessen · **`Ganzzahl`** statt **`int`** · Parametertyp vergessen: **`int summe(a, b)`** · Reihenfolge vertauscht

Parameter, Argumente und lokale Variablen

```
int helm = 10;
int s = schutzwert(helm + 2, 3);
```

Argumente: Werte beim Aufruf



Parameter stehen im Kopf, **Argumente** beim Aufruf. Anzahl, Reihenfolge und Typ müssen passen.

Gültigkeit: Wo lebt eine Variable?

```
int schutzwert(int ruestung, int stufe) {
    int wert = ruestung + 5 * stufe;
    return wert;
}
```

ruestung, stufe, wert: nur hier, nur während des Aufrufs

```
int s = schutzwert(12, 3);
println(wert);    // Fehler: cannot find symbol
for (int i = 0; i < 3; i++) { }
println(i);       // Fehler: cannot find symbol
```

Gleicher Name in zwei Methoden = zwei verschiedene Variablen. Ergebnis nach außen nur per `return`.

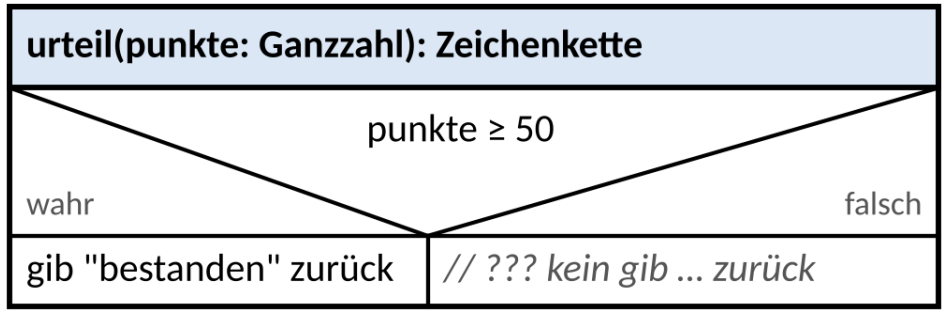
return: jeder Weg braucht eine Rückgabe

FALSCH

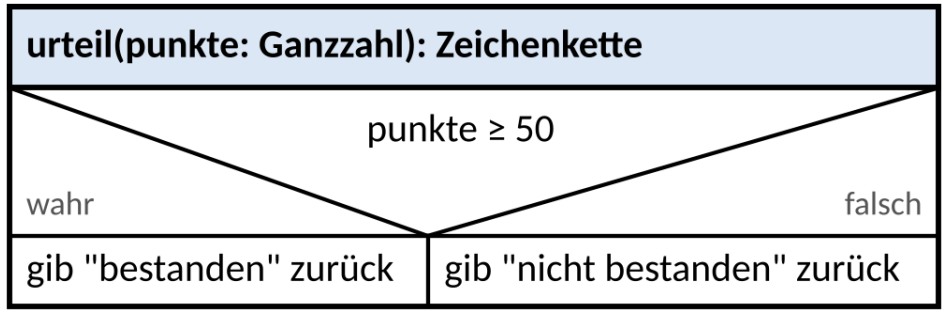
```
public static String urteil(int punkte) {
    if (punkte >= 50) {
        return "bestanden";
    }
}   // Fehler: missing return statement
```

RICHTIG

```
public static String urteil(int punkte) {
    if (punkte >= 50) {
        return "bestanden";
    } else {
        return "nicht bestanden";
    }
}
```



falsch-Weg endet ohne „gib … zurück“



jeder Weg endet mit „gib … zurück“

`return` beendet sofort. Was danach steht, läuft nie: **„unreachable statement“**

void oder Rückgabewert: was macht der Aufrufer?

nichts = void: tut etwas, liefert nichts

```
public static void zeigeLeben(String name, int leben) {
    println(name + ": " + leben + " LP");
}
```

```
zeigeLeben("Aria", 80);
```

```
Aria: 80 LP
```

Konsole

Fehler: `int x = zeigeLeben("Aria", 80);` – void kann man nicht speichern.

Tipp: `return;` ohne Wert beendet eine void-Methode vorzeitig. Ein Wert hinter return ist hier verboten.

Rückgabewert verwenden: drei Wege

1. **speichern**

```
int s = schutzwert(12, 3);
```

2. **ausgeben**

```
println(schutzwert(12, 3));
```

3. **weiterrechnen**

```
if (schutzwert(12, 3) >= 30) {…}
```

Falle: `schutzwert(12, 3);` allein kompiliert – aber das Ergebnis 27 verpufft. Wer etwas liefert, dessen Wert muss man auffangen.

Methode ruft Methode: das Aufrufprotokoll

```
public static int schutzwert(int ruestung, int stufe) {
    int wert = ruestung + 5 * stufe;
    return wert;
}

public static boolean haeltStand(int schaden,
                                 int ruestung, int stufe) {
    int schutz = schutzwert(ruestung, stufe);
    return schutz >= schaden;
}
```

Merke: Der Aufrufer wartet, bis die aufgerufene Methode ihren Wert liefert. Dann geht es genau an der Aufrufstelle weiter.

Zerlegen lohnt: haeltStand rechnet nicht selbst, sondern nutzt schutzwert.

Hauptprogramm: ok = haeltStand(30, 12, 3)
wartet … dann ok = false

ruft auf

haeltStand: schaden = 30, ruestung = 12, stufe = 3
schutz = 27; 27 >= 30 → gib false zurück

ruft auf

schutzwert: ruestung = 12, stufe = 3
wert = 27 → gib 27 zurück

Aufruf	lokale Variable	Rückgabe
`haeltStand(30, 12, 3)`	`schutz = 27`	**`false`**
`schutzwert(12, 3)`	`wert = 27`	**27**

Vorhersage: Was gibt das Programm aus?

```
public static void heile(int leben) {
    leben = leben + 20;
}

public static void heileAlle(int[] team) {
    for (int i = 0; i < team.length; i++) {
        team[i] = team[i] + 20;
    }
}

// Hauptprogramm
int held = 50;
int[] gruppe = {50, 30};
heile(held);
heileAlle(gruppe);
println(held);         // ?
println(gruppe[0]);    // ?
```

Team-Tipp (1 Minute, ohne Gerät)

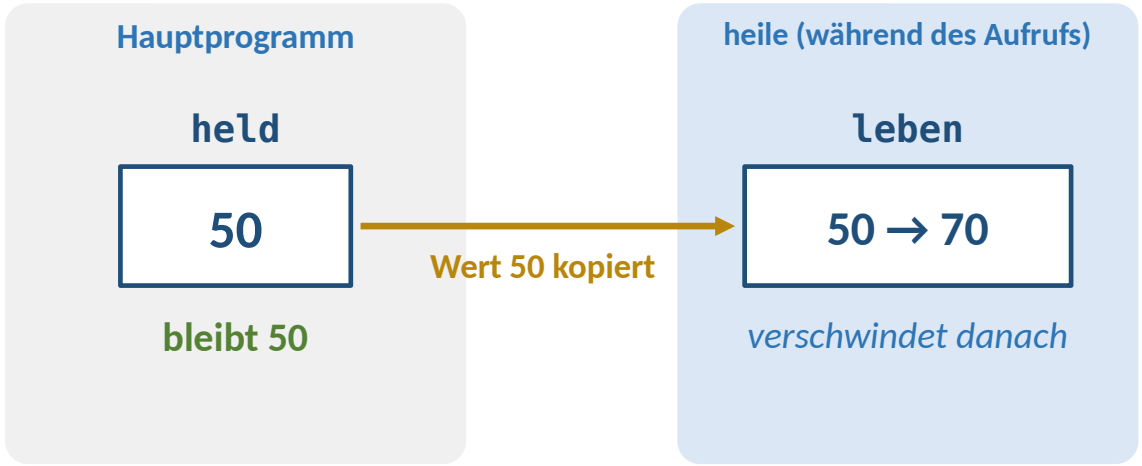
- **A** 50 und 50
- **B** 70 und 70
- **C** 50 und 70
- **D** 70 und 50



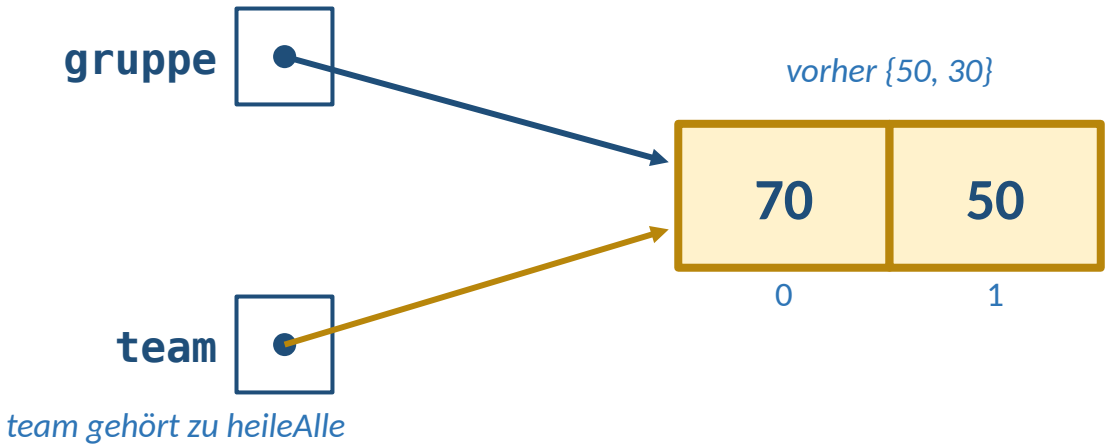
Begründet euren Tipp:
Was bekommt heile übergeben – und was heileAlle?

Auflösung: Kopie des Werts – Kopie des Pfeils

`heile(held)` int: Kopie des Werts



`heileAlle(gruppe)` Reihung: Kopie des Pfeils



Antwort C

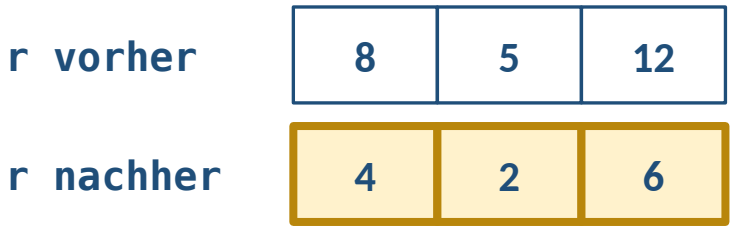
```
println(held)  →  50
println(gruppe[0])  →  70
```

Merke: Java übergibt immer eine Kopie. Bei `int` ist das der Wert – die Methode kann die Variable des Aufrufers nie ändern. Bei Reihungen ist es der Pfeil: Beide zeigen auf dasselbe Regal, Änderungen am Inhalt wirken nach außen.

Reihung verändern oder neue Reihung liefern?

halbiereAlle(r: Reihung von Ganzzahl): nichts

```
public static void halbiereAlle(int[] r) {
    for (int i = 0; i < r.length; i++) {
        r[i] = r[i] / 2;
    }
}
```

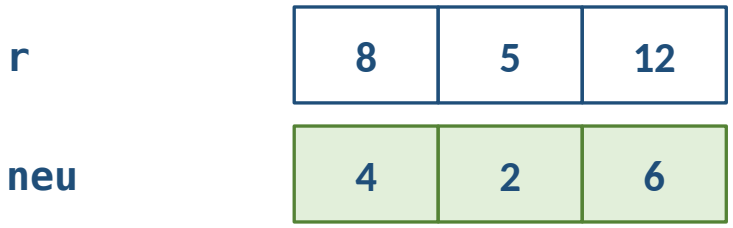


in-place: r selbst ändert sich

Die Signatur verrät es: Rückgabe nichts → in r arbeiten.
Rückgabe Reihung → mit **`new`** neu anlegen.

halbierteKopie(r: Reihung von Ganzzahl): Reihung von Ganzzahl

```
public static int[] halbierteKopie(int[] r) {
    int[] neu = new int[r.length];
    for (int i = 0; i < r.length; i++) {
        neu[i] = r[i] / 2;
    }
    return neu;
}
```



neue Reihung, r bleibt unverändert

Falle: **`int[] neu = r;`** ist keine Kopie – danach ist auch r halbiert: {4, 2, 6}.

Hilfsmethoden: große Aufgaben zerlegen

```java
public static boolean istVokal(char c) {
    return c == 'a' || c == 'e' || c == 'i'
        || c == 'o' || c == 'u' || c == 'A'
        || c == 'E' || c == 'I' || c == 'O'
        || c == 'U';
}

public static int anzahlVokale(String s) {
    int anzahl = 0;
    for (int i = 0; i < s.length(); i++) {
        if (istVokal(s.charAt(i))) {
            anzahl++;
        }
    }
    return anzahl;
}
```



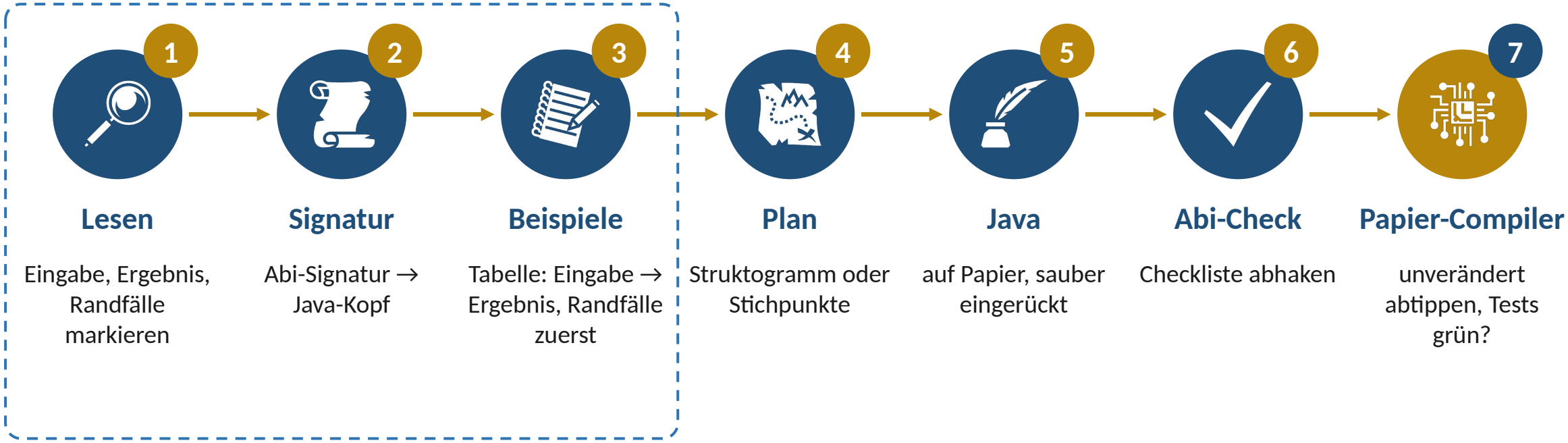
`anzahlVokale("Akademie")` → 5

Werkzeugkiste aus Level 0: einmal schreiben, oft aufrufen

- `summe(r)`
- `maximum(r)`
- `zaehle(r, x)`
- `indexVon(r, x)`

Achtung Abi: „bereits implementiert“ heißt: aufrufen, nicht neu schreiben.

Das Rezept: von der Aufgabe zur Methode



Vorarbeit: liefert Leerabfrage und Grenzen gratis

Zeitbudget Abi: ≈ 2,5 min je BE
5 BE ≈ 12 min – davon 3 min für die Schritte 1–3

Papier-Compiler 0: Methode auf AB 5 von Hand, dann unverändert abtippen. Alle Tests grün beim 1. Versuch? Dann „compiliert“ deine Handschrift.

Vorgemacht (1): letzterIndexVon

AUFGABE (Beispiel) · 4 BE ≈ 10 min

Die Operation letzterIndexVon liefert den **Index** des **letzten Vorkommens** des Werts x in der Reihung r. Ist x **nicht in r enthalten**, wird **−1** geliefert.

letzterIndexVon(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl

Implementieren Sie die Operation letzterIndexVon.

1 Lesen und markieren

Eingabe: Reihung r, Wert x
Ergebnis: ein Index (Ganzzahl)
Randfälle: x fehlt → −1 · r leer → −1 · x mehrfach → letztes

2 Signatur → Java-Kopf

```
public static int letzterIndexVon(int[] r, int x)
```

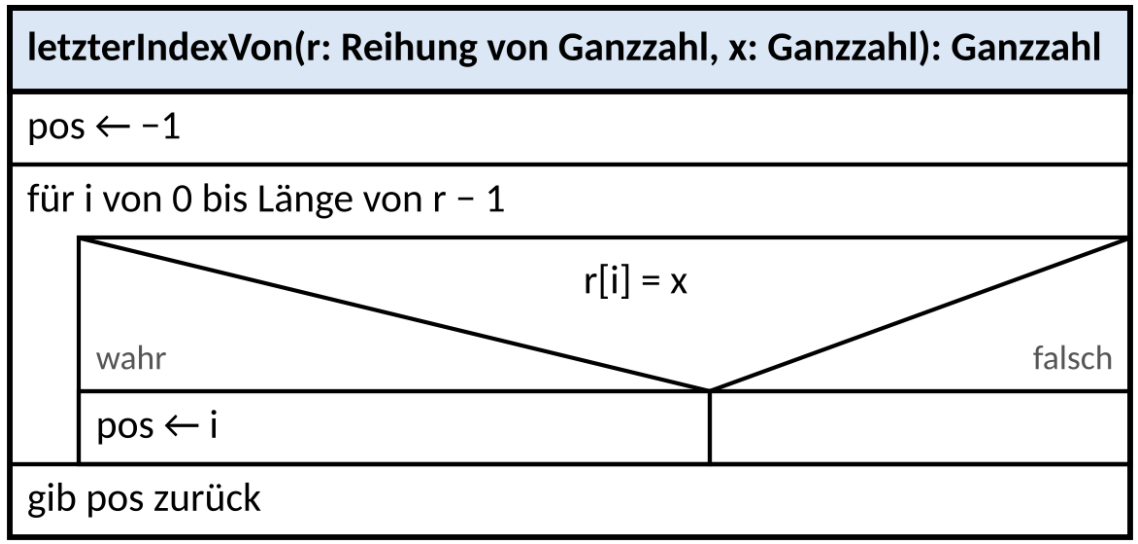
3 Beispiele – Randfälle zuerst

r	x	Ergebnis	Warum?
{4, 9, 4, 7}	4	**2**	letztes von zwei
{4, 9, 4, 7}	7	**3**	ganz am Ende: Grenze!
{4, 9, 4, 7}	5	**-1**	nicht enthalten
{}	4	**-1**	leere Reihung

Idee: Von vorn nach hinten laufen und jeden Treffer merken – der zuletzt gemerkte ist der letzte. Startwert −1 deckt „nicht enthalten“ und „leer“ ab.

Vorgemacht (2): Plan, Java, Abi-Check

4 Struktogramm



5 Java auf Papier

```java
public static int letzterIndexVon(int[] r, int x) {
    int pos = -1;
    for (int i = 0; i < r.length; i++) {
        if (r[i] == x) {
            pos = i;
        }
    }
    return pos;
}
```

6 Abi-Check

- Kopf wie Signatur: Name, Reihenfolge, Typen
- return in jedem Pfad – genau eins, am Ende
- Leerabfrage nötig? Nein: kein Zugriff auf r[0], Schleife läuft 0-mal
- Grenzen: i von 0 bis r.length − 1
- kein return in der Schleife – sonst wäre es das erste Vorkommen
- r wird nicht verändert

Fehlergalerie: die Top 6 der Akademie

1 Fehlender Rückgabepfad

```
if (n > 0) {
    return 1;
}  // und dann?
```

Folge: Compiler: missing return statement

Fix: **`else-Zweig oder return am Ende`**

2 Off-by-one

```
for (int i = 0;
     i <= r.length; i++)
```

Folge: Absturz beim Index r.length

Fix: **`i < r.length`**

3 = statt ==

```
if (gefunden = true) {
    …
```

Folge: Zuweisung: immer wahr

Fix: **`if (gefunden)`** **`bzw.`** **`==`**

4 Ganzzahlige Division

```
double d = summe / r.length;
```

Folge: 7 / 3 ergibt 2.0 statt 2.33…

Fix: **`(double) summe / r.length`**

5 Rückgabe zu früh

```
if (r[i] == x) { return true; }
else { return false; }
```

Folge: nur r[0] wird geprüft

Fix: **`return false nach der Schleife`**

6 Reihung verändert statt Kopie

```
int[] neu = r;
neu[i] = r[i] / 2;
```

Folge: Original wird mit verändert

Fix: **`int[] neu = new int[r.length];`**

Abi-Format und Abi-Check

SO KLINGT ES IM ABITUR

„Implementieren Sie die Methode …“
Java mit Methodenkopf nach Signatur

„Entwickeln Sie ein Struktogramm …“
Kopf mit Signatur, ←, „gib … zurück“

„Geben Sie den Methodenkopf an.“
nur der Java-Kopf, kein Rumpf

„Dokumentieren Sie … in einer Tracetabelle.“
eine Zeile je Durchlauf bzw. Aufruf

„Erläutern Sie … / Begründen Sie …“
in ganzen Sätzen, mit Bezug zum Code

Jede Teilaufgabe nennt ihre BE: ≈ 2,5 min je BE.

Abi-Check vor der Abgabe

- Signatur wie gefordert: Name, Reihenfolge, Typen, Rückgabetyp
- return in jedem Pfad – auch nach der Schleife
- vor jedem Zugriff: Leer- bzw. Indexabfrage
- Schleifengrenzen: 0 bis length − 1
- Original unverändert, falls eine neue Reihung verlangt ist
- nur erlaubte Zeichenketten-Operationen, Vergleich mit equals
- unsichere Stellen kurz kommentieren (//)

Boss 0: Die Aufnahmeprüfung der Akademie



DS 7

60 Minuten · 24 BE
Zeitbudget wie im Abi: ≈ 2,5 min pro BE

Papier, ohne Gerät
erlaubt sind nur deine Werkzeugkarten

Neuer Kontext
gleiches Handwerk wie in der Akademie – Transfer

Zusätzlich im Lab: Online-Boss
„Aufnahmeprüfung: Die Bibliothek der Akademie“ · 25 min

Vorbereitung: Warm-up-Karten W0-09 bis W0-22 · Werkzeugkarten · Abi-Check